An Always-On Language Model with a Learned Output Router: Routing Thought, Speech, Writing, Code and Action from a Single Stream

Siavash Shirzadeh Barough

Preprint

October 2026

Abstract

Large language models (LLMs) are usually run as turn-based generators that stop at an end-of-sequence (EOS) token and are wrapped in external agent loops that decide when to think, speak, call a tool, or wait. We propose and test a simpler organisation, inspired by the functional layout of the brain, in which sensory inputs, an ongoing stream of thought and several executive outputs share one continuous workspace, and a selection mechanism decides, token by token, which output a computation drives. A decoder-only LLM emits a single token stream; a small learned *router head* assigns every generated token to one of seven channels (think, say, write, code, exec, mem, idle); EOS is replaced by an *idle* channel, so the model never terminates and waits for the next event; external events (user messages, tool results, images) are inserted into the stream at the time they arrive, including mid-thought; and tokens routed to a *mem* channel are retained when old context is evicted. We fine-tuned Qwen3-0.6B by imitation of scripted policies in AsyncWorld, a synthetic environment with concurrent tasks, deadlines, cancellations, revisions and asynchronous tools, and compared the router with three control protocols trained on identical episodes: in-stream channel tags, a rule-based orchestrator that delivers events only at line boundaries, and a turn-based agent loop (3 seeds per arm, 300 test episodes per split). The router matched tags in distribution (task success 99.6% vs. 99.4%) and was slightly more accurate on a heavier, out-of-distribution load (+0.9 pp, 95% CI [+0.5, +1.4] over episodes; smaller than the seed-to-seed spread), while saving the tokens that tags spend on channel switches. Compared with the orchestrator, it answered deadline-bound requests more often in distribution (+6.9 pp) with fewer tokens, but under the heavier load overall task success was similar and whole-episode success was lower (-4.0 pp, CI [-7.3, -0.8]). The turn-based loop, which by construction cannot respond while blocked, met few deadlines (11.7% vs. 98.4%). In a multimodal pilot with a frozen CLIP-ConvNeXt image encoder and self-distilled GSM8K rationales routed to the think channel, one router model answered 95.3% of visual questions on held-out synthetic scenes and solved 52.0% of GSM8K problems, comparable to the base model's built-in thinking mode (53.7%); urgent requests arriving mid-thought were answered on time (97.3%) but reduced math accuracy by -6.3 pp (CI [-10.7, -2.0]). Retaining mem-routed tokens did not change long-horizon success, as the environment rarely required information older than the recent context window. These small-scale, synthetic results show that output routing, waiting and interruption handling can be learned inside the model rather than hard-coded around it; they do not show benefits on natural tasks or at scale.

1 Introduction

A human does not alternate between a "thinking program" and a "speaking program" orchestrated from outside. Perception, ongoing thought and action run concurrently, and a person can stop mid-thought to answer a question, resume, wait, or remember to finish a task later. Current LLM agents obtain similar behaviour through scaffolding: the model generates until EOS, and an external controller parses the output, routes it to tools or the user, decides when the model may continue, and re-prompts it with new observations [18, 23, 33]. This scaffolding is hand-written, turn-based, and blind to the model's internal state, so the model cannot be interrupted between turns and cannot keep thinking while it waits.

We ask whether this control can be moved inside the model with minimal machinery. Our proposal has four components:

1. **One stream, many outputs.** The model emits one token per tick. A linear router head on the final hidden state predicts the output channel of each token (think, say, write, code, exec, mem, idle). Channel identity is fed back through a learned channel embedding, so the model knows which "effector" produced each past token.
2. **No stop token.** EOS is removed from the action space and replaced by an *idle* channel. Idle advances the clock to the next event; the model never terminates.
3. **Event-timed inputs.** User messages, tool results and images are inserted into the stream at the tick at which they arrive, even in the middle of a line of thought, and training sequences contain such insertions, so the model learns to react to them without an external controller.
4. **Memory by routing.** Tokens routed to the mem channel are retained when old context is evicted, giving a learned, content-addressed retention policy under a fixed context budget.

The design is motivated by a coarse functional analogy to the brain (Section 3): a shared workspace that broadcasts to many specialised processors [1, 6], a selection system that gates one action among competing ones [16, 21], intrinsic activity that persists without external input [20], and a separate system for retaining selected episodes [15]. We use the analogy to generate design hypotheses, not as a claim of biological fidelity.

Throughout, time is measured in *ticks*: one generated token takes one tick, inserted input tokens take none, and waiting (idle) advances the clock to the next event; latency is the number of ticks between a request's arrival and its answer. Training data come from an *oracle*, a hand-written scripted policy for each control protocol, whose own score gives an upper reference ("oracle ceiling") for a model that imitates it.

Contributions. (i) An architecture and training recipe for an always-on LLM with a learned per-token output router, an idle channel in place of EOS, event-timed input insertion, and mem-routed retention. (ii) AsyncWorld, a controlled benchmark with concurrent tasks, deadlines, cancellations, revisions and asynchronous tools, together with scripted oracle policies for four control protocols. (iii) A comparison with hypotheses specified before the main runs against in-stream tags, a rule-based orchestrator and a turn-based loop at equal data, with three seeds per arm and paired bootstrap CIs. (iv) A multimodal pilot in which vision and step-by-step math reasoning share the same routed stream.

2 Related work

Continuous and asynchronous LLM agents. The closest work trains a single think/say/tool stream for continuous-time agents, but an external orchestrator of about 200 lines "alone knows the clock", forces the end of thoughts, and injects events at interrupt points as chat turns [11]. Multi-Stream LLMs decode several role-fixed streams in lockstep with per-stream positions and stream embeddings; streams are fixed rather than learned [26]. Asynchronous Reasoning lets a thinker and a writer share one KV cache, with the writer gated by an inserted probe every paragraph, training-free [30]. Asynchronous function calling [7, 25] and interruptibility benchmarks [29, 37] study how agents handle tools and users that change their minds. Parallel decoding methods (Hogwild!, Multiverse) run several reasoning threads concurrently [22, 32]. Our design differs in that channel selection is a learned per-token decision on one stream, termination is replaced by a learned idle action, and inputs are inserted at their arrival tick without a controller.

Full-duplex dialogue. Spoken dialogue models listen and speak simultaneously [5, 14, 28, 36], and SHANKS lets a spoken model think while listening [3]. A recent duplex agent uses a learned per-second control token (listen/speak/interrupt) and delegates tasks to a separate agent [17]. These systems route between at most a few speech states at a coarse time step; we route every token among seven text-level channels including thought, code and memory.

Latent and hidden reasoning. Pause tokens [9], Quiet-STaR [35] and continuous latent reasoning [10] separate internal computation from output. STaR bootstraps rationales from a model's own correct answers [34]; we use the same idea to obtain think-channel content.

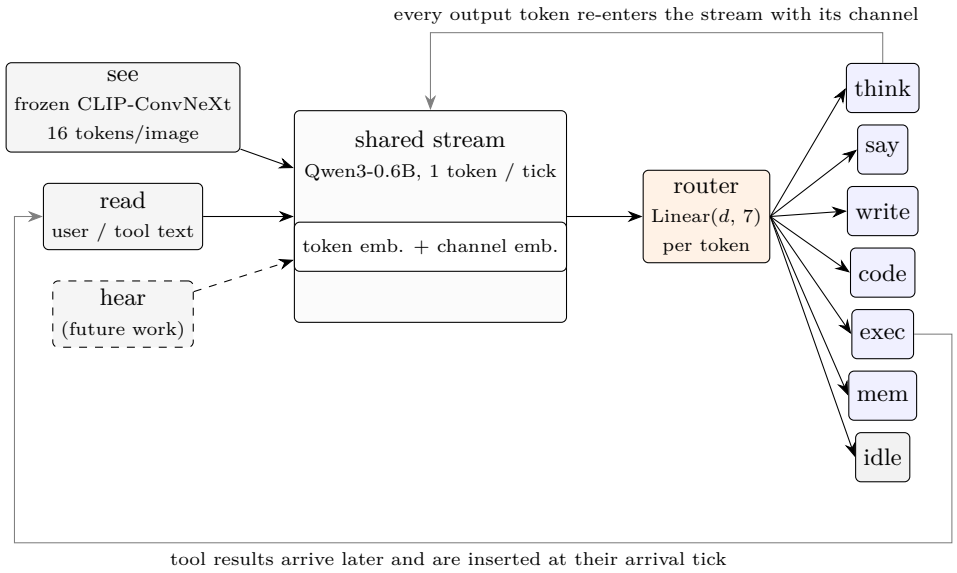


Figure 1: Architecture. Senses write tokens into one stream; the router assigns each generated token to an output channel; all tokens, including inputs inserted mid-thought, are fed back with a channel embedding. There is no stop token: idle waits for the next event, and mem-routed tokens are kept when old context is evicted.

Workspace models in deep learning. Global-workspace ideas have been proposed for deep networks [27] and implemented as a shared bottleneck among neural modules [8]. Our model uses a pretrained LLM as the shared workspace and adds a router as the selection mechanism.

Memory. MemGPT manages memory through explicit function calls issued by the model and executed by a controller [18]. Our mem channel is an output like any other; retention is a side effect of routing.

3 A brain-inspired organisation

Figure 1 and Table 1 summarise the analogy that guided the design. We list the neuroscience findings that motivated each component and state explicitly what we did and did not implement. The analogy is functional and coarse; we do not claim that the network computes as the brain does.

4 Method

Model. Let $x_{1:t}$ be the token stream and $c_{1:t}$ the channel of each token: an output channel (think, say, write, code, exec, mem or idle) for generated tokens, or an input channel (sys, user, tool or see) for inserted tokens. The input at position i is $e(x_i) + g(c_i)$, where e is the pretrained token embedding and g a learned channel embedding initialised to zero, so training starts from the unmodified pretrained model. A decoder-only transformer produces h_t; the LM head gives $p(x_{t+1} \mid h_t)$ and the router gives $p(c_{t+1} \mid h_t) = \text{softmax}(W_r h_t)$, $W_r \in \mathbb{R}^{7\times d}$. At inference both are decoded greedily; if the router selects idle, no content token is emitted.

Clock, idle and events. One generated token consumes one tick. Functionally, idle is an EOS that the model itself resumes from: in a controller-based system the same behaviour requires stopping generation and re-prompting when the next event arrives, which is what the turn-based and orchestrator baselines do; we therefore did not run a separate idle-versus-EOS ablation. Each event has an arrival tick; when the clock reaches it, the event's tokens are inserted into the stream (one per forward step, taking no model ticks) before the model generates again. An idle step advances the clock to the next pending event. An episode ends when the environment has no further events and all tasks are resolved, or at a step cap; the model itself never stops.

Table 1: Functional analogy used as a design guide. "Implemented" describes this paper's model; the right-hand column lists the source of the motivating finding.

Brain system	Motivating finding	Implemented here
Sensory cortices	Specialised encoders feed a common representational space	Frozen CLIP-ConvNeXt image encoder [13, 19] projected to 16 stream tokens; text inputs on user/tool channels; hearing not implemented
Global workspace	Information becomes globally available to many processors via a shared, limited-capacity workspace [1, 6]	One causal token stream that every input writes into and every output reads from
Basal ganglia	Focused selection of one action and inhibition of competitors [16, 21]	Router head selecting one channel per token; idle acts as "no action"
Intrinsic activity	The brain is active in the absence of tasks; a default mode of function [20]	No EOS; the model idles and resumes rather than terminating
Hippocampal system	Selected episodes are stored separately from the slowly changing cortical store [15]	Mem-routed tokens survive context eviction
Prefrontal control	Design assumption (no specific finding tested): ongoing thoughts can be interrupted and resumed when urgent stimuli arrive	Event-timed insertion; training traces in which thoughts are suspended and resumed

Outputs. Each channel keeps its own line buffer; a completed line on say, write or code is a user-visible answer; a completed exec line is a tool call (calc, lookup, or run code) whose result returns after a sampled latency; think and mem lines are internal.

Training objective. We fine-tune on oracle traces with $\mathcal{L} = \sum_t \mathbb{1}[c_t \neq \text{idle}]\,\mathrm{CE}(x_t) + \sum_t \mathrm{CE}(c_t)$ over generated positions only; inserted input tokens are context, not targets.

Memory under eviction. With a context cap of 1024 tokens, when the history exceeds the cap the model keeps the system prompt, the most recent 640 tokens and, under the *mem* policy, up to 192 earlier tokens that were routed to mem. The *plain* policy keeps only the recent window. Training includes long episodes viewed through both policies.

5 AsyncWorld and baselines

Tasks. Each episode samples concurrent tasks arriving at random ticks: *calc* (arithmetic via the calc tool), *lookup* (a code word via the lookup tool), *write* (sort numbers, answered on write), *code* (write a one-line Python expression on code and execute it), and *quick* (a one-step sum that must be answered on say within 24 ticks). Cancellations ("cancel Tn") must be honoured within a 10-tick grace; revisions ("Tn change 88 to 24") require the corrected answer. Tool latencies are 10–50 ticks in distribution (*id*: 3–5 tasks, 300 episodes), 10–60 under *stress* (6–8 tasks with more cancellations and revisions, out of the training distribution, 300 episodes), and 20–150 in *long* episodes (28–32 tasks over 3000 ticks, 32 episodes).

Oracle and protocols. A scripted policy produces training traces for four protocols on identical episodes. Each protocol has its own oracle so every arm imitates the best policy it can express:

- **Router (ours):** channels by router head; events inserted at their arrival tick; thoughts preempted by urgent events; partially written lines suspended and resumed for a quick task.
- **In-stream tags:** the same delivery and policy, but channel switches are special tokens (e.g. `<|think|>`) in the text. This isolates the channel-selection mechanism (router head and channel embedding vs. tag tokens) from asynchronous delivery.

Table 2: Main results (mean over training seeds, number in column 2; 95% bootstrap CI over episodes). Success in %; latency in ticks (median per episode); tokens generated per episode. Grey rows: the scripted oracle for the same protocol.

Arm	Seeds	Task	Episode	Quick	Cancel	Revise	Quick lat.	Tokens
In-distribution (300 episodes, 3–5 tasks)								
Router (ours)	3	99.6 [99.3, 99.8]	97.4 [96.0, 98.7]	98.4 [97.4, 99.3]	100.0 [100.0, 100.0]	100.0 [100.0, 100.0]	7 [7, 8]	294 [286, 302]
oracle	–	99.5	97.0	98.2	99.3	100.0	7	293
In-stream tags	3	99.4 [99.1, 99.7]	96.4 [94.6, 98.1]	98.2 [97.0, 99.1]	99.3 [97.8, 100.0]	99.7 [99.1, 100.0]	9 [8, 9]	320 [312, 327]
oracle	–	99.6	97.7	98.7	99.3	100.0	9	318
Orchestrator	3	97.6 [96.9, 98.3]	87.1 [83.3, 90.6]	91.5 [88.6, 94.1]	100.0 [100.0, 100.0]	100.0 [100.0, 100.0]	11 [10, 12]	359 [349, 370]
oracle	–	97.5	86.7	91.2	100.0	100.0	11	361
Turn-based	3	66.5 [64.9, 68.1]	4.3 [2.3, 6.7]	11.7 [8.5, 15.2]	1.4 [0.0, 3.5]	99.7 [99.1, 100.0]	206 [193, 220]	338 [329, 347]
oracle	–	66.4	4.3	11.7	0.7	100.0	207	338
Stress (300 episodes, 6–8 tasks)								
Router (ours)	3	97.7 [97.2, 98.1]	81.8 [78.6, 84.9]	99.4 [98.9, 99.8]	96.8 [95.6, 97.9]	95.2 [93.9, 96.4]	7 [7, 7]	515 [504, 527]
oracle	–	99.9	99.3	99.8	99.7	100.0	7	513
In-stream tags	3	96.8 [96.2, 97.3]	77.0 [73.7, 80.2]	99.6 [99.3, 99.9]	97.2 [96.1, 98.1]	93.7 [92.4, 95.0]	9 [8, 9]	546 [537, 555]
oracle	–	99.9	99.3	99.8	100.0	100.0	9	556
Orchestrator	3	98.0 [97.6, 98.4]	85.8 [82.8, 88.8]	98.1 [97.0, 99.1]	97.8 [96.9, 98.6]	98.0 [97.2, 98.8]	8 [8, 9]	628 [613, 645]
oracle	–	99.5	96.0	98.2	100.0	100.0	8	619
Turn-based	3	60.3 [59.4, 61.2]	0.0 [0.0, 0.0]	6.3 [4.4, 8.5]	0.0 [0.0, 0.0]	100.0 [100.0, 100.0]	487 [461, 515]	660 [650, 670]
oracle	–	60.3	0.0	6.3	0.0	100.0	487	659

- **Orchestrator:** events are delivered only at line boundaries, or during a thought, which is then abandoned and restarted, mimicking an external controller that preempts generation [11].
- **Turn-based:** one user message at a time; tool calls block until the result returns; equivalent to a ReAct-style loop [33].

Oracle ceilings are reported for every arm. The turn-based ceiling is low by construction on deadline tasks; it measures what that protocol can express, not a weakness of a particular implementation.

6 Experimental setup

Base model: Qwen3-0.6B [31] (Apache-2.0 licence), full fine-tuning, bf16 autocast with fp32 master weights, AdamW (learning rate 5×10^{-5}; 10^{-3} for new parameters), cosine schedule, one epoch over 12,000 id episodes plus 600 long episodes $\times$ 4 eviction views (7.4–8.3 M tokens per arm), 16k tokens per batch, one NVIDIA RTX 4090 (24 GB) per run (about 13 min per run; all experiments used under 30 GPU-hours). No hyperparameters were tuned for any arm. Every arm was trained with three seeds. Evaluation was closed-loop and greedy, with batched rollouts in which each sequence advances exactly one token per step. Metrics: task success; episode success (all tasks correct); success by category (quick, cancel, revise, plain); median quick latency; generated tokens. Statistics: mean over seeds of the per-episode mean; 95% CIs from 2000 bootstrap resamples of test episodes, paired across arms because all arms see the same episodes. The hypotheses (H1–H4 below) and these choices were written down before the main results (`paper/method_notes.md`); they were not registered externally. The bootstrap CIs reflect sampling of test episodes, not variation across training seeds; we therefore also report per-seed values. The confirmatory family was the six in-distribution comparisons of task and quick-task success between the router and each comparator, which we report with Holm correction; other p-values are uncorrected, and and stress, latency, token and multimodal analyses should be read as exploratory.

Hypotheses. **H1**: with asynchronous delivery fixed, the router matches or beats in-stream tags with fewer tokens and lower latency. **H2**: the router beats the orchestrator on quick tasks and latency. **H3**: router and tags beat the turn-based loop on interrupt categories. **H4**: under eviction, mem-routed retention beats a plain sliding window on revisions and cancellations.

7 Results

H1, router vs. tags. In distribution, the router and tags reached the same success (Table 2; Δtask +0.2 pp, CI [-0.2, +0.5]; Holm-adjusted $p = 0.750$). Under stress the router was more accurate (Δtask +0.9 pp, CI

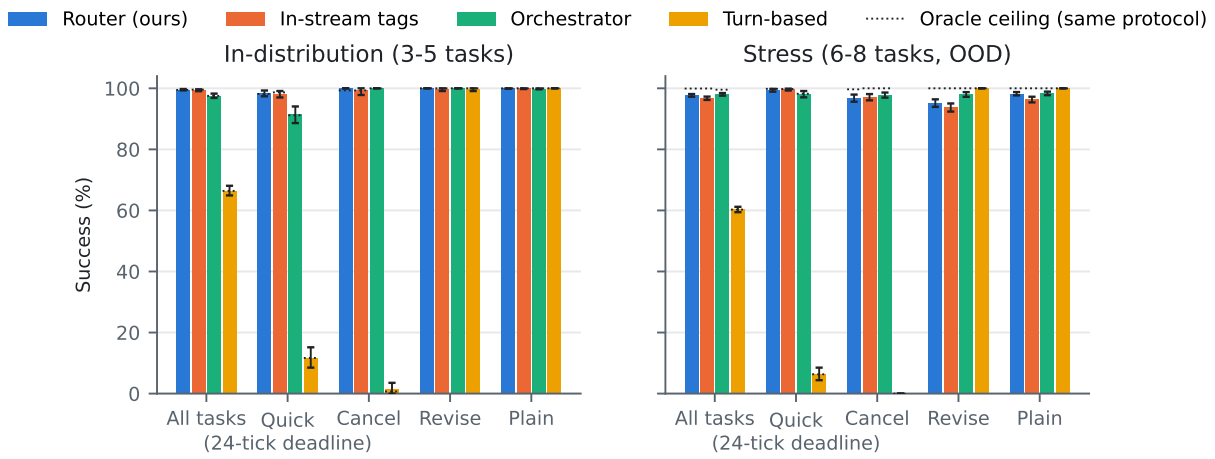


Figure 2: Success by category. Bars: mean over training seeds; error bars: 95% bootstrap CI over 300 episodes; dotted lines: oracle ceiling for that protocol.

Table 3: Paired differences, router minus comparator (pp = percentage points; latency in ticks; tokens per episode). 95% CI and two-sided bootstrap p over the same 300 episodes.

Split	Router minus	ΔTask (pp)	ΔEpisode (pp)	ΔQuick (pp)	ΔQuick lat.	ΔTokens
id	In-stream tags	+0.2 [-0.2, +0.5]; p=0.401	+1.0 [-1.1, +3.0]; p=0.394	+0.2 [-1.2, +1.6]; p=0.809	-1.3 [-1.7, -0.8]; p=<0.001	-25.3 [-27.2, -23.7]; p=<0.001
id	Orchestrator	+2.0 [+1.3, +2.7]; p=<0.001	+10.3 [+6.8, +14.2]; p=<0.001	+6.9 [+4.3, +9.8]; p=<0.001	-3.6 [-4.3, -2.9]; p=<0.001	-65.1 [-69.7, -60.7]; p=<0.001
id	Turn-based	+33.1 [+31.5, +34.8]; p=<0.001	+93.1 [+90.3, +95.6]; p=<0.001	+86.7 [+83.3, +89.9]; p=<0.001	-199.2 [-212.5, -185.8]; p=<0.001	-43.7 [-47.6, -40.0]; p=<0.001
id	Router w/o channel emb.	+0.1 [-0.1, +0.2]; p=0.404	+0.4 [-0.7, +1.4]; p=0.481	-0.1 [-0.6, +0.4]; p=0.903	+0.0 [-0.1, +0.2]; p=0.749	+0.1 [-0.2, +0.5]; p=0.616
stress	In-stream tags	+0.9 [+0.5, +1.4]; p=<0.001	+4.8 [+1.6, +7.9]; p=0.004	-0.2 [-0.8, +0.4]; p=0.489	-1.6 [-1.7, -1.4]; p=<0.001	-31.2 [-39.5, -20.6]; p=<0.001
stress	Orchestrator	-0.3 [-0.8, +0.1]; p=0.160	-4.0 [-7.3, -0.8]; p=0.022	+1.3 [+0.3, +2.3]; p=0.010	-1.4 [-1.7, -1.1]; p=<0.001	-113.3 [-129.2, -99.3]; p=<0.001
stress	Turn-based	+37.4 [+36.5, +38.3]; p=<0.001	+81.8 [+78.2, +84.9]; p=<0.001	+93.1 [+90.9, +95.1]; p=<0.001	-479.8 [-504.7, -453.9]; p=<0.001	-145.0 [-153.3, -134.4]; p=<0.001
stress	Router w/o channel emb.	+0.2 [-0.2, +0.6]; p=0.435	-0.6 [-3.2, +2.1]; p=0.739	+0.1 [-0.3, +0.5]; p=0.563	-0.0 [-0.1, +0.0]; p=0.724	-12.1 [-27.1, +2.7]; p=0.106

[+0.5, +1.4]; Δepisode +4.8 pp, CI [+1.6, +7.9]) and in both splits it generated fewer tokens (Δ -25.3 and -31.2 tokens per episode) and answered quick tasks earlier (Δ median latency -1.3 and -1.6 ticks; Table 3). The token saving equals the channel-switch tags the tag model must spell out. Across training seeds (Table 7), the standard deviation of stress task success was 0.9 pp for the router and 1.5 pp for tags, as large as the difference, so the stress advantage should be read as tentative. H1 was supported for tokens and latency; success was equivalent in distribution and marginally higher under stress.

H2, router vs. orchestrator. In distribution the router beat the orchestrator on quick tasks (Δ +6.9 pp, CI [+4.3, +9.8]) and overall (Δ +2.0 pp, CI [+1.3, +2.7]) because the orchestrator can only react at line boundaries or by abandoning a thought. Under stress, quick-task success still favoured the router (Δ +1.3 pp, CI [+0.3, +2.3]) and the router used -113.3 fewer tokens per episode, but overall task success did not differ (Δ -0.3 pp, CI [-0.8, +0.1]) and episode success (all tasks of an episode correct) favoured the orchestrator (Δ -4.0 pp, CI [-7.3, -0.8]). The orchestrator restarts abandoned thoughts from the beginning, which costs tokens but rarely leaves a half-finished line on a channel. Seed-level results (Table 7) agree that the two protocols are close under stress. H2 was supported for latency and quick tasks, not for overall success under stress.

H3, vs. turn-based. The turn-based loop solved plain tasks and revisions but could not meet deadlines or honour cancellations that arrived while it was blocked (quick success 11.7% in distribution, 6.3% under stress, close to its oracle ceiling of 11.7% and 6.3%). H3 was supported by a wide margin; the size of the gap is a property of the protocol and the environment's deadlines rather than of model quality.

Ablation: channel embedding. Removing the channel embedding, so that the model sees past tokens without their channel, did not change success (Δtask +0.1 pp, CI [-0.1, +0.2]in distribution; Table 3). In AsyncWorld, channel identity can apparently be inferred from content (for example the "Tn:" prefix and line structure), so this ablation does not establish that the embedding is necessary.

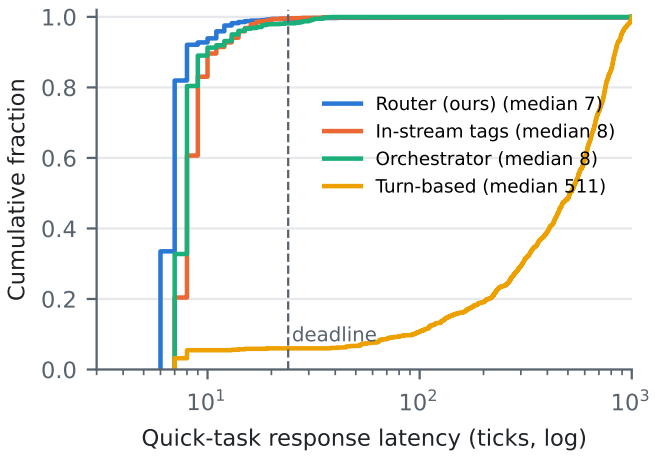


Figure 3: Quick-task response latency under stress (all seeds; unanswered tasks at the right edge).

Table 4: Long-horizon episodes (32 episodes, about 30 tasks over 3000 ticks) under three context policies; mean of 3 seeds with 95% CI.

Arm	Eviction	Seeds	Task	Episode	Revise	Cancel	Plain	Tokens
Router (ours)	none (full context)	3	84.3 [82.7, 86.1]	20.8 [12.5, 31.2]	78.5 [75.4, 82.0]	74.9 [71.4, 78.8]	87.3 [85.2, 89.4]	4578 [4416, 4678]
Router (ours)	keep mem-routed	3	97.8 [97.0, 98.5]	50.0 [37.5, 62.5]	97.3 [94.8, 99.2]	96.1 [93.3, 98.4]	98.1 [96.8, 99.1]	2481 [2434, 2533]
Router (ours)	sliding window	3	97.7 [97.0, 98.4]	49.0 [36.5, 62.5]	97.9 [95.3, 99.7]	96.0 [93.2, 98.4]	98.1 [97.0, 99.0]	2506 [2451, 2562]
In-stream tags	none (full context)	3	91.9 [89.8, 93.7]	14.6 [7.3, 24.0]	93.4 [90.7, 95.9]	95.9 [92.6, 98.4]	90.0 [87.9, 92.1]	2910 [2648, 3257]
In-stream tags	keep mem-routed	3	95.0 [91.8, 97.6]	44.8 [30.2, 59.4]	94.6 [91.0, 97.8]	97.3 [95.1, 99.2]	94.2 [89.8, 97.7]	2853 [2650, 3106]
In-stream tags	sliding window	3	96.3 [94.0, 98.1]	53.1 [38.5, 66.7]	96.2 [93.4, 98.4]	97.7 [95.8, 99.3]	96.2 [93.0, 98.4]	2792 [2634, 3004]

H4, memory by routing. Table 4 and Figure 4 report the long-horizon split (32 episodes of about 30 tasks, 1024-token cap). H4 was not supported: for the router, keeping mem-routed tokens and a plain sliding window gave the same task success (Δ +0.1 pp, CI [-0.6, +0.7]; revisions -0.6 pp, CI [-1.7, +0.0]; cancellations +0.2 pp, CI [-0.6, +0.8]), and for tags the plain window was slightly better (Δ -1.3 pp, CI [-2.5, -0.4]). A post-hoc stratification explains the null result: revisions and cancellations arriving more than 200 ticks after their task were solved as often as those arriving sooner under both policies (router, 99.0% for both policies, $n = 297$), so the information needed was still inside the 640-token recent window. AsyncWorld therefore did not stress retention, and with 32 episodes the test of H4 was inconclusive rather than a refutation. An unplanned finding was that *not* evicting hurt: with the full, unbounded context (up to about 9k tokens, beyond the $\leq$1.2k-token training sequences) router task success fell to 84.3% from 97.8%, and the model generated almost twice as many tokens (4578 vs. 2481). We attribute this to a train/test length mismatch rather than to the method: without length generalisation or longer training sequences, an always-on model needs a bounded working context.

Multimodal pilot: seeing and reasoning in the same stream. To test whether diverse cognitive content can share the routed stream, we trained an additional router model on a mixture of AsyncWorld episodes, visual questions and GSM8K problems [4]. Images (synthetic scenes of 1–6 coloured circles, squares and triangles) were encoded by a frozen CLIP-ConvNeXt-Base [13, 19] trained on LAION-2B [2, 24]; the 8×8 feature map was pooled to 16 tokens, mapped by a trained two-layer projector [12], and inserted on a *see* channel. Visual questions asked for counts by colour or shape, existence, or the colour of a unique shape; the oracle wrote a brief think line and the answer on say. For math, we generated rationales from the base model (non-thinking mode) and kept one correct rationale per training problem (4622 of 7473), following STaR [34] without its rationalisation step (problems with no correct sample were dropped); the rationale was routed to think and the final number to say. Training followed the two-stage recipe of LLaVA [12]: stage 1 trained only the new parameters (projector, visual position, channel embedding, router) with the LLM frozen on 30,000 vision-only episodes of four questions each (6.7 M tokens); stage 2 fine-tuned all parameters on 12,000 mixed AsyncWorld episodes (with up to one math problem and two images each) plus

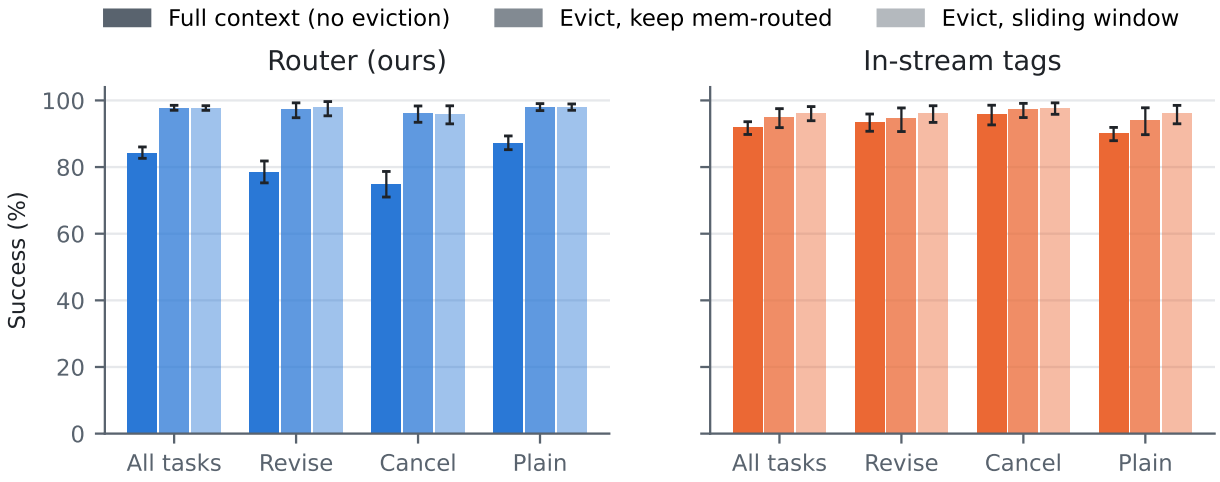


Figure 4: Long-horizon success by context policy.

Table 5: Multimodal pilot (one seed per model). Accuracy in %, bootstrap 95% CI over items. Left: two-stage training (projector alignment, then full fine-tuning). Right: ablation trained without the alignment stage and with fewer visual examples.

Evaluation	n	router_mm (two-stage)	no alignment stage
GSM8K, base Qwen3-0.6B, non-thinking mode	300	38.3 [33.0, 43.3]	–
GSM8K, base Qwen3-0.6B, thinking mode	300	53.7 [47.7, 59.3]	–
GSM8K, think channel, no interruptions	300	52.0 [46.7, 57.7]	49.7 [44.0, 55.3]
GSM8K, 2 quick tasks arrive mid-thought	300	45.7 [40.0, 51.0]	46.3 [40.7, 52.0]
quick tasks within deadline	600	97.3 [96.0, 98.5]	96.5 [95.0, 97.8]
Visual QA, all questions	600	95.3 [93.7, 96.8]	42.2 [38.2, 46.3]
count by colour	150	94.0 [90.0, 97.3]	42.7 [34.7, 50.7]
count by shape	152	88.8 [83.6, 93.4]	32.9 [25.7, 40.1]
existence	158	98.7 [96.8, 100.0]	69.6 [62.7, 77.2]
colour of unique shape	140	100.0 [100.0, 100.0]	20.7 [14.3, 27.9]
concurrent non-visual tasks	900	99.6 [99.1, 99.9]	99.3 [98.8, 99.8]
AsyncWorld id episodes, all tasks	1637	99.8 [99.5, 99.9]	99.7 [99.4, 99.9]

15,000 vision episodes with concurrent tasks (11.2 M tokens). A first model trained with stage 2 only, on the mixed episodes alone (12,000 visual questions), is reported as an ablation. To check that the frozen features contained the needed information, we fitted linear probes on the 16 pooled feature vectors of the 20,000 training images; on the 1000 test images they predicted per-colour and per-shape object counts with 92–97% accuracy (majority baseline 37–43%). At test time (first 300 GSM8K test problems; 300 vision episodes with two images each on held-out scenes) we measured math accuracy with no interruptions and with two quick tasks arriving 15–120 ticks after the problem, that is, typically mid-thought, which the model must answer within the deadline before resuming. Results are in Table 5.

With two-stage training the single router model answered 95.3% of visual questions on held-out scenes (existence 98.7%, colour of a unique shape 100.0%, counting by colour 94.0%, counting by shape 88.8%) while solving 99.6% of the concurrent non-visual tasks in the same episodes. Without the alignment stage, visual accuracy was 42.2% and the colour question was at chance (20.7% vs. 25%), although the frozen features supported 92–97% probe accuracy; the encoder was therefore not the limiting factor; because the ablation also had fewer visual examples, we cannot separate the effect of the alignment stage from that of more visual data. On GSM8K, routing self-distilled rationales to the think channel gave 52.0% without interruptions, compared with 38.3% for the base model in non-thinking mode and 53.7% in its built-in thinking mode (up to 1536 generated tokens; 22% of outputs without a parseable answer); the router did not differ from thinking

mode (paired Δ -1.7 pp, CI [-8.0, +5.0]) while using a median of 116 think tokens per problem. When two quick tasks arrived during the thought, the model answered 97.3% of them within the deadline and resumed, but math accuracy fell to 45.7% (paired difference -6.3 pp, CI [-10.7, -2.0]; $n = 300$). Pausing and resuming a train of thought therefore had a measurable cost at this scale. AsyncWorld performance remained high with the added modalities (99.8% task success in distribution; text-only router 99.6%, different seeds and task mixture).

8 Discussion

The results support a narrow claim: in a controlled environment, a small LLM can learn to select its own output channel token by token, to wait without terminating, and to handle interruptions at their arrival time, with no external controller and with task success close to its scripted teacher in distribution. A learned router head was as accurate as spelling channel tags into the text and avoided their token overhead. Relative to a rule-based orchestrator, in-model control mainly bought latency and tokens; overall success under heavy load was similar. The largest effect, against a turn-based loop, follows from what that protocol can express. The multimodal pilot showed that a visual sense and multi-step reasoning can be added to the same routed stream without degrading the agentic behaviour, provided the visual features are first aligned; it also exposed a cost of interrupting reasoning that a brain-like system would need to manage. The memory channel did not help in our environment, which leaves its value open.

What the analogy suggests next. The brain analogy points to experiments we did not run: (i) more senses (audio frames inserted on a hear channel at their arrival time; video as a sequence of image events); (ii) learned rather than scripted policies, by reinforcement learning on environment reward, since imitation caps performance at the oracle; (iii) richer think content (planning, hypothesis testing, critique) from stronger teachers; (iv) parallel effectors, for example speaking while writing, which a single stream can only interleave; (v) consolidation, in which mem-routed content is written to weights or an external store.

9 Limitations

- **Synthetic tasks and imitation.** AsyncWorld is small and scripted; all arms imitate scripted oracles; models came within about a point of their oracle ceilings on task success in distribution, but under stress the router's episode success was 81.8% against an oracle ceiling of 99.3%; differences between arms partly reflect the design of each protocol's oracle rather than learned strategies.
- **Scale.** One 0.6B model, one epoch, three to five seeds per arm; the multimodal model has one seed. Effects may differ at scale.
- **Missing baselines.** We did not compare against published agent systems [11] on their benchmarks, did not ablate idle against EOS, and did not train a matched plain-SFT model on the same GSM8K rationales.
- **Baselines.** The orchestrator and turn-based arms are our implementations of these control styles, trained on the same data; they are not the published systems [11, 33].
- **Oracle corrections.** During the study we found that the oracle could suspend a say line for a quick answer on the same channel, which merged two lines in about 12% of router and tags episodes. We fixed the oracle, rebuilt the data and retrained all affected arms; superseded checkpoints and runs were archived. A related resume-order error in the multimodal oracle was fixed before that model was trained.
- **Interrupted reasoning.** Suspending a math rationale for urgent requests reduced GSM8K accuracy by -6.3 pp; the oracle resumes the thought verbatim, and the model may need explicit training to re-establish context after a pause.
- **Memory.** AsyncWorld rarely required information older than the recent window, so the mem-retention hypothesis was not tested in a regime where it could matter; the retention budget and window were fixed, not tuned; long-horizon results rest on 32 episodes.

- **Brain analogy.** The mapping in Table 1 is a design heuristic. We did not compare model activity with neural data.

10 Ethics and broader impact

All data were synthetic or public (GSM8K, MIT licence [4]); no human participants or personal data were involved. An always-on model that acts without an external controller moves safety-relevant decisions (when to call a tool, when to act) into learned weights; deployments would still need external permission checks and sandboxing of tool execution, which this design does not replace.

11 Conclusion

Replacing the stop token with a learned idle action, routing every token to an output channel with a small head, and inserting events at their true arrival time let a single LLM think, speak, write, code, call tools, remember and wait without an agent loop. In controlled synthetic tests this matched in-stream tags with fewer tokens, answered urgent requests faster than a rule-based orchestrator at similar overall success, and, as expected by construction, handled interrupts far better than a turn-based loop. The multimodal pilot shows that vision and step-by-step reasoning can share the same routed stream. Whether the approach helps on natural, open-ended tasks remains to be tested.

Reproducibility. Environment, oracle, data builder, training, rollout and analysis code, the hypotheses document, all test-episode seeds and per-episode result files are available at `https://github.com/SiavashShirzad/NeverIdleMind`.

References

[1] Bernard J. Baars. *A Cognitive Theory of Consciousness*. Cambridge University Press, 1988.

[2] Mehdi Cherti, Romain Beaumont, Ross Wightman, et al. Reproducible scaling laws for contrastive language-image learning, 2023.

[3] Cheng-Han Chiang, Xiaofei Wang, Linjie Li, Chung-Ching Lin, Kevin Lin, Shujie Liu, Zhendong Wang, Zhengyuan Yang, Hung-yi Lee, and Lijuan Wang. SHANKS: Simultaneous hearing and thinking for spoken language models, 2025.

[4] Karl Cobbe, Vineet Kosaraju, et al. Training verifiers to solve math word problems, 2021.

[5] Alexandre Défossez, Laurent Mazaré, Manu Orsini, Amélie Royer, Patrick Pérez, Hervé Jégou, Edouard Grave, and Neil Zeghidour. Moshi: a speech-text foundation model for real-time dialogue, 2024.

[6] Stanislas Dehaene, Michel Kerszberg, and Jean-Pierre Changeux. A neuronal model of a global workspace in effortful cognitive tasks. *Proceedings of the National Academy of Sciences*, 95(24):14529–14534, 1998. doi: 10.1073/pnas.95.24.14529.

[7] In Gim, Seung-seob Lee, and Lin Zhong. Asynchronous LLM function calling, 2024.

[8] Anirudh Goyal, Aniket Didolkar, et al. Coordination among neural modules through a shared global workspace, 2021.

[9] Sachin Goyal, Ziwei Ji, Ankit Singh Rawat, Aditya Krishna Menon, Sanjiv Kumar, and Vaishnavh Nagarajan. Think before you speak: Training language models with pause tokens, 2023.

[10] Shibo Hao, Sainbayar Sukhbaatar, DiJia Su, Xian Li, Zhiting Hu, Jason Weston, and Yuandong Tian. Training large language models to reason in a continuous latent space, 2024.

[11] Bojie Li and Noah Shi. Never stop thinking: Continuous-time language agents, 2026.

[12] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning, 2023.

[13] Zhuang Liu, Hanzi Mao, et al. A convnet for the 2020s, 2022.

[14] Ziyang Ma, Yakun Song, Chenpeng Du, Jian Cong, Zhuo Chen, Yuping Wang, Yuxuan Wang, and Xie Chen. Language model can listen while speaking, 2024.

[15] James L. McClelland, Bruce L. McNaughton, and Randall C. O'Reilly. Why there are complementary learning systems in the hippocampus and neocortex: insights from the successes and failures of connectionist models of learning and memory. *Psychological Review*, 102(3):419–457, 1995. doi: 10.1037/0033-295X.102.3.419.

[16] Jonathan W. Mink. The basal ganglia: focused selection and inhibition of competing motor programs. *Progress in Neurobiology*, 50(4):381–425, 1996. doi: 10.1016/S0301-0082(96)00042-1.

[17] Orantqing, Shengpeng Ji, Junlong Tong, Jialong Zuo, Dongjie Fu, Di Cao, Yangzhuo Li, Shangda Wu, Franz, Evan, Theron Veyra, Changhao Pan, Jingyu Lu, Dongchao Yang, Zhifei Xie, Yang Tan, Xiaoyu Shen, Xiaoda Yang, Wenfu Wang, Teddy Sun, Steve Yves, and Zhou Zhao. Multimodal duplex interaction agent, 2026. Author names as listed on arXiv; "Franz" and "Evan" appear as single names.

[18] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems, 2023.

[19] Alec Radford, Jong Wook Kim, et al. Learning transferable visual models from natural language supervision, 2021.

[20] Marcus E. Raichle, Ann Mary MacLeod, Abraham Z. Snyder, William J. Powers, Debra A. Gusnard, and Gordon L. Shulman. A default mode of brain function. *Proceedings of the National Academy of Sciences*, 98(2):676–682, 2001. doi: 10.1073/pnas.98.2.676.

[21] Peter Redgrave, Tony J. Prescott, and Kevin Gurney. The basal ganglia: a vertebrate solution to the selection problem? *Neuroscience*, 89(4):1009–1023, 1999. doi: 10.1016/S0306-4522(98)00319-4.

[22] Gleb Rodionov, Roman Garipov, Alina Shutova, George Yakushev, Erik Schultheis, Vage Egiazarian, Anton Sinitsin, Denis Kuznedelev, and Dan Alistarh. Hogwild! inference: Parallel LLM generation via concurrent attention, 2025.

[23] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools, 2023.

[24] Christoph Schuhmann, Romain Beaumont, Richard Vencu, et al. LAION-5B: An open large-scale dataset for training next generation image-text models. NeurIPS 2022 Datasets and Benchmarks, 2022.

[25] Kou Shi, Ziao Zhang, Shiting Huang, Avery Nie, Zhen Fang, Qiuchen Wang, Lin Chen, Huaian Chen, Zehui Chen, and Feng Zhao. AsyncTool: Evaluating the asynchronous function calling capability under multi-task scenarios, 2026.

[26] Guinan Su, Yanwu Yang, Xueyan Li, and Jonas Geiping. Multi-stream LLMs: Unblocking language models with parallel streams of thoughts, inputs and outputs, 2026.

[27] Rufin VanRullen and Ryota Kanai. Deep learning and the global workspace theory. *Trends in Neurosciences*, 44(9):692–704, 2021. doi: 10.1016/j.tins.2021.04.005.

[28] Bandhav Veluri, Benjamin N Peloquin, Bokai Yu, Hongyu Gong, and Shyamnath Gollakota. Beyond turn-based interfaces: Synchronous LLMs as full-duplex dialogue agents, 2024.

[29] Tsung-Han Wu, Mihran Miroyan, David M. Chan, Trevor Darrell, Narges Norouzi, and Joseph E. Gonzalez. Are large reasoning models interruptible?, 2025.

[30] George Yakushev, Nataliia Babina, Masoud Vahid Dastgerdi, Vyacheslav Zhdanovskiy, Denis Kuznedelev, Alina Shutova, and Max Ryabinin. Asynchronous reasoning: Training-free interactive thinking LLMs, 2025. v3, May 2026.

[31] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report, 2025.

[32] Xinyu Yang, Yuwei An, Hongyi Liu, Tianqi Chen, and Beidi Chen. Multiverse: Your language models secretly decide how to parallelize and merge generation, 2025.

[33] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models, 2022.

[34] Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. STaR: Bootstrapping reasoning with reasoning, 2022.

[35] Eric Zelikman, Georges Harik, Yijia Shao, Varuna Jayasiri, Nick Haber, and Noah D. Goodman. Quiet-STaR: Language models can teach themselves to think before speaking, 2024.

[36] Xinrong Zhang, Yingfa Chen, Shengding Hu, Xu Han, Zihang Xu, Yuanwei Xu, Weilin Zhao, Maosong Sun, and Zhiyuan Liu. Beyond the turn-based game: Enabling real-time conversations with duplex models, 2024.

[37] Henry Peng Zou, Chunyu Miao, Wei-Chieh Huang, Yankai Chen, Yue Zhou, Hanrong Zhang, Yaozu Wu, Liancheng Fang, Zhengyao Gu, Zhen Zhang, Kening Zheng, Fangxin Wang, Yi Nian, Shanghao Li, Wenzhe Fan, Langzhou He, Weizhi Zhang, Xue Liu, and Philip S. Yu. When users change their mind: Evaluating interruptible agents in long-horizon web navigation, 2026.

A Example trace

Table 6 shows the first in-distribution test episode for router seed 0, as generated in closed loop. Consecutive tokens with the same source and channel are merged; ↩ marks a newline, which completes a line on that channel. Inputs are inserted wherever they arrive: the cancellation of T2 interrupts a thought, the requests for T4 and T5 arrive in the middle of think lines, and the quick task T6 arrives in the middle of a mem line, which the model suspends, answers on say, and then completes.

B Seed-level results

C Training details

Table 6: Closed-loop trace (router, seed 0, test episode 0).

Source	Channel	Tokens
model	idle	<|idle|>
input	user	T1 quick: 9+5? ↩
model	say	T1: 14 ↩
model	idle	<|idle|>
input	user	T2 ask: what is the code for kiwi? ↩
model	think	T2 needs the code for kiwi. That needs the lookup tool; I will report it when the
input	user	cancel T2 ↩
model	think	The user cancelled T2
input	user	T3 ask: what is (83+81)*18? ↩
model	think	. I drop it and will ignore any result for it. ↩
model	mem	T2 cancelled ↩
model	think	T3
input	user	T4 code: count multiples of 7 below 17 ↩
model	think	wants (
input	user	T5 write: sort 49 91 87 73 54 ↩
model	think	83+81)*18. I should not do this in my head, so I will call calc and wait for the result. ↩
model	mem	T3 open calc (
input	user	T6 quick: 1+3? ↩
model	say	T6: 4 ↩
model	mem	83+81)*18 ↩
model	exec	T3 calc (83+81)*18 ↩
model	think	T4 needs the count multiples of 7 below 17. I will write one line of python, run it, and say the output. ↩
model	mem	T4 open code ↩
model	code	T4: print(len([i for i in range(1
input	tool	T3 calc (83+81)*18 = 2952 ↩
model	code	,17) if i%7==0])) ↩
model	exec	T4 run ↩
model	say	T3: 2952 ↩
model	mem	T3 closed ↩
model	think	T5 is a sorting job for 49 91 87 73 5
input	tool	T4 output: 2 ↩
model	think	4. No tool needed; put them in increasing order and write them down. ↩
model	write	T5: 49 54 73 87 91 ↩
model	mem	T5 closed ↩
model	say	T4: 2 ↩
model	mem	T4 closed ↩
model	idle	<|idle|>

Table 7: Task success per training seed and across seeds (mean, SD and t-based 95% CI over seeds).

Split	Arm	Seeds	Task success per seed (%)	Mean ± SD	95% CI (seeds, t)
id	Router (ours)	3	99.4, 99.5, 99.7	99.6 ± 0.2	[99.2, 99.9]
id	In-stream tags	3	99.3, 99.5, 99.5	99.4 ± 0.1	[99.1, 99.7]
id	Orchestrator	3	97.7, 97.5, 97.6	97.6 ± 0.1	[97.3, 97.9]
id	Turn-based	3	66.4, 66.5, 66.5	66.5 ± 0.0	[66.4, 66.5]
id	Router w/o channel emb.	3	99.2, 99.6, 99.6	99.5 ± 0.3	[98.8, 100.1]
stress	Router (ours)	3	98.6, 96.7, 97.8	97.7 ± 0.9	[95.4, 100.0]
stress	In-stream tags	3	98.4, 95.5, 96.3	96.8 ± 1.5	[93.1, 100.4]
stress	Orchestrator	3	97.5, 98.1, 98.5	98.0 ± 0.5	[96.7, 99.4]
stress	Turn-based	3	60.3, 60.3, 60.3	60.3 ± 0.0	[60.3, 60.3]
stress	Router w/o channel emb.	3	97.2, 97.6, 97.7	97.5 ± 0.3	[96.8, 98.2]

Table 8: Training and evaluation settings (identical for all arms unless noted).

Base model	Qwen3-0.6B, full fine-tuning, tied embeddings
New parameters	channel embedding (zero init), router Linear$(1024, 7)$; tag tokens for tag arms
Optimiser	AdamW, $\beta = (0.9, 0.95)$, no weight decay, lr 5×10^{-5} (new parameters 10^{-3})
Schedule	30-step warm-up, cosine decay, 1 epoch
Batch	length-bucketed, 16,384 tokens (multimodal: 12,288)
Precision	fp32 master weights, bf16 autocast, gradient checkpointing
Training data	12,000 id episodes + 600 long episodes $\times$ 4 eviction views; 7.4–8.3 M tokens per arm
Context	cap 1024 tokens; recent window 640; mem budget 192
Decoding	greedy; step cap 3000 (9000 for long episodes)
Hardware	one NVIDIA RTX 4090 per run; about 13 min per training run